

Animation in CSS

Part 1: What is an Animation? (The First Principle)

- **The Fundamental Truth:** An animation is a **change in style over a period of time**.
- **The Analogy: A Sunset**
 - "Think about a sunset. At 6 PM, the sky is bright blue. This is the **start state**."
 - By 7 PM, the sky is a deep orange. This is the **end state**."
 - The sunset itself is the **animation**—the gradual, smooth change from blue to orange over the course of one hour (the **duration**)."
- **The Core Problem:** A normal CSS rule, like `.sky { background-color: blue; }`, only defines a single moment in time. How do we describe the entire sunset from start to finish?
- **The Logical Solution: A Two-Part System**

"CSS solves this by giving us a two-part system:"

 1. **The Storyboard (@keyframes):** "First, we describe the key moments of our story. We define what the sky looks like at the beginning and at the end. This 'storyboard' is called a **@keyframes** rule."
 2. **The Director (animation property):** "Second, we tell an element (our 'sky') to perform this story. We use the **animation** property to give it directions, like how long the sunset should take."

Part 2: Creating the Storyboard with @keyframes

- **The First Principle:** An animation is defined by its key states. The simplest animation has a beginning and an end.

- **The Syntax:** code CSS

```
@keyframes animation-name {  
  from { /* Start styles */ }  
  to { /* End styles */ }  
}
```

- **@keyframes:** The special command to start defining an animation.
 - **animation-name:** A name you invent for your storyboard.
- **Practical Example: The "Sunset" Animation** code CSS

Let's create the storyboard for our sunset. We will animate the background-color.

```
@keyframes sunset-effect {  
  from {  
    background-color: #87CEEB; /* A bright Sky Blue */  
  }  
  to {  
    background-color: #FF4500; /* A deep OrangeRed */  
  }  
}
```

- **Explain:** "This storyboard is named sunset-effect. It tells a simple story: start as sky blue, end as orange-red. The browser will automatically figure out all the in-between colors to make the change smooth."

Part 3: Applying the Animation with the animation Property

- **The First Principle:** A storyboard needs an element to apply it to.
- **The Core Problem:** How do we tell our <div> to use the sunset-effect storyboard, and how long should it take?
- **The Solution:** The animation property (and its individual parts).

- **Building it up Step-by-Step:** code Html code CSS

```
<div class="sky"></div>
```

```
/* Don't forget to include the @keyframes rule from above! */

.sky {
  height: 200px;
  width: 200px;
  background-color: #87CEEB; /* The starting color */

  /* --- Let's give our director the instructions --- */

  /* 1. Which storyboard to use? (Required) */
  animation-name: sunset-effect;

  /* 2. How long should the animation take? (Required) */
  animation-duration: 5s; /* 5 seconds */
}
```

- **Result (Live Demo):** When the page loads, the box will start as sky blue and smoothly change to orange-red over 5 seconds. But then it stops.

Part 4: Controlling the Animation's Repetition

- **The Problem:** The animation only plays once. How do we make it loop or go back and forth?
- **The Solution:** Introduce two new "director's instructions."
 1. **animation-iteration-count (How Many Times?):**
 - Controls how many times the animation repeats.
 - **infinite:** The keyword for a loop that never ends.

```
.sky {  
  /* ... other animation properties ... */  
  animation-iteration-count: infinite;  
}
```

- **Result:** The box will animate from blue to orange, then instantly **snap back** to blue and start over, forever. This snap is jarring.

2. animation-direction (How to Loop Smoothly?):

- **The Problem:** The "snap back" at the end of the loop doesn't look like a real sunset and sunrise. We need it to animate backwards smoothly.
- **The Solution:** The alternate value.
- **alternate:** This tells the animation to play forwards (from → to) on the first run, then backwards (to → from) on the second run, and so on.
- **Example:** code CSS

```
.sky {  
  /* ... other animation properties ... */  
  animation-iteration-count: infinite;  
  animation-direction: alternate;  
}
```

- **Result:** A perfect day/night cycle. The box will smoothly animate from blue to orange (the sunset), and then smoothly back from orange to blue (the sunrise), forever.

Part 5: Animating Size - A Practical "Progress Bar" Project

This project introduces animating a different property, width, and the concept of what happens after an animation ends.

- **The Goal:** Create a bar that fills up from left to right, once.
- **HTML:** A container <div> and a fill <div>. code Html

```
<div class="progress-container">
  <div class="progress-fill"></div>
</div>
```

- **The CSS Storyboard:** code CSS

```
@keyframes fill-the-bar {
  from {
    width: 0%;
  }
  to {
    width: 100%;
  }
}
```

- **The CSS Director's Instructions:** code CSS

```
.progress-fill {
  height: 30px;
  background-color: #4CAF50; /* Green */
  width: 0; /* Important: It starts at 0 width */

  animation-name: fill-the-bar;
  animation-duration: 4s;

  /* What happens when it's done? */
  animation-fill-mode: forwards;
}
```

- **Introducing animation-fill-mode:**
 - **The Problem:** By default, once our 4-second animation is over, the .progress-fill element will snap back to its original style (width: 0;). The bar

would fill up and then instantly become empty again.

- **The Solution:** `animation-fill-mode: forwards;` This is a crucial instruction that tells the browser: "After the animation is finished, **keep the styles from the final (to) keyframe.**"
- **Result:** The progress bar animates from 0% to 100% width and then **stays full.**

fill-mode	Behavior During delay	Behavior After Animation Ends	Use Case
none (Default)	Shows element's resting style.	Reverts to element's resting style.	Simple looping animations where the start/end states match.
forwards	Shows element's resting style.	Retains the last keyframe's style.	Animations that need to "stick" in their final state (e.g., a fade-out).
backwards	Applies the first keyframe's style.	Reverts to element's resting style.	Animations with a delay that need a specific starting state (e.g., a fade-in).
both	Applies the first keyframe's style.	Retains the last keyframe's style.	The "all-in-one" solution for delayed, one-shot animation